Designing, decomposing, and delivering systemic software

Source Code Foundations

SEAtS | 892971

University of Southampton

Intended Learning Outcomes

By the end you will be able to:

1. Explain why source code acts as a living design document, using systemic design mindsets (holism, feedback loops, giga-maps) to keep requirements and code aligned.
2. Apply Krischer et al.'s informed decomposition (cohesion, coupling, interface contracts) to split firmware → gateway → cloud → frontend systems into independently evolvable modules.
3. Evaluate readability, coding standards (Google, PEP 8, MISRA C), linting, and error-handling models that keep codebases auditable.
4. Justify dependency, version-control, and CI/CD gate choices as governance that preserves traceability from requirements to deployment.

© University of Southampton

Agenda

1. Source code as a living design document & systemic design mindsets.
2. Informed decomposition and the language/abstraction stack.
3. Readability, coding standards, and linting.
4. Coupling, cohesion, interface contracts, and error-handling models.
5. Performance trade-offs and design patterns for modularity.
6. Dependencies, version control workflows, and CI/CD gates.

© University of Southampton

Source Code as Living Design Document

Source code is the “blueprint + assembly manual” combined—every change should update both the map and the build instructions.

Code acts as the executable specification—requirements, architecture, and deployment assumptions converge inside each module.

Treat repositories like living system maps: docstrings, READMEs, and ADRs narrate stakeholder intent and boundary decisions.

Traceability tags (requirement IDs, test IDs) keep the living document aligned with audits, CI gates, and regulatory expectations.

© University of Southampton

Systemic Design Mindsets

Holism: When refactoring a module, consider upstream requirements, downstream data pipelines, and socio-technical impacts.

Feedback loops: Monitoring dashboards act as feedback channels; connect alerts back to backlog items for rapid adjustments.

Giga-maps & causality diagrams: Visualise requirements ↔ components ↔ test links so architectural shifts stay transparent.

Note: Causality diagrams show reinforcing or balancing loops; each arrow represents how one subsystem amplifies or dampens another.

© University of Southampton

From Idea to Source Code – System Journey



- Embed “traceability breadcrumbs” at each arrow (user stories, architecture decisions, code comments, test IDs).
- Innovation labs mirror CI/CD loops—each build collects stakeholder feedback before feeding the next concept.

© University of Southampton

Informed Decomposition

Structural metrics: Treat excessive coupling like a stiff mechanical joint; the more resistance between modules, the harder it is to evolve safely.

Purposeful slicing: Define component boundaries so teams optimise independently (similar to microservices or subsystems) while staying aligned to system objectives.

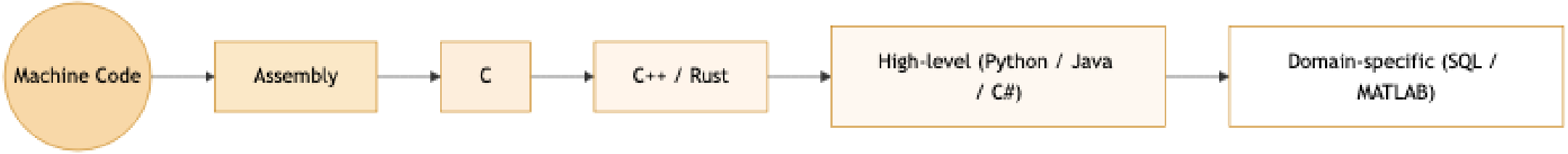
Interface discipline: Independent optimisation depends on explicit API contracts, shared schemas, and transparent assumptions.

Risk: Poor decomposition creates cascading failures—e.g., a firmware change silently breaking cloud analytics because interface expectations lived only in developer heads.

© University of Southampton

Programming Languages & Building Blocks

Layer	Typical Language	Role in Requirements Traceability	Notes
Firmware	C/C++	Deterministic timing for sensor specs	Constraints: memory, energy
Gateway	C++/Rust/Python	Protocol translation, buffering	Needs configurable retries & validation
Cloud services	Python/Node/Go	Analytics, APIs, ML flows	Emphasise CI, scaling, observability
Frontend	TypeScript	Visualization, stakeholder feedback loops	Accessibility & user telemetry



Each language is a specialized dialect in a multi-lingual negotiation—translation layers (APIs) prevent miscommunication.

© University of Southampton

Building Blocks Refresher + Practice

- **Variables & state:** Trace how state flows from sensor registers to UI state stores.
- **Control flow & exceptions:** Map where requirements demand deterministic decisions vs. recoverable errors.
- **Modules & packages:** Align each module with a requirement cluster (e.g., “REQ-3: secure data path”).

```
module SensorGateway
    function buffer(sample)
        assert sample conforms SchemaV2
        queue.enqueue(sample, retry_policy = EXPONENTIAL)
    end
end
```


© University of Southampton

Project Structure & Traceability Hooks

```
project/
  ├─ specs/requirements/REQ-*.md  ← stakeholder + non-functional context
  ├─ docs/gigamaps/               ← systemic diagrams, causality notes
  ├─ src/                         ← implementation modules (tagged w/ REQ IDs)
  ├─ interfaces/                  ← API schemas, protocol definitions
  ├─ tests/unit|integration|hil/  ← verification layers
  ├─ ci/                          ← pipelines, quality gates
  └─ ops/                         ← deployment, monitoring scripts
```


© University of Southampton

Readability vs Ambiguity

```
# Ambiguous naming + magic numbers
if status == 2 then send(x)
```

```
# Intent-rich alternative
if sensorPacket.isValidChecksum() then
    telemetryChannel.transmit(sensorPacket.withTraceId(traceId))
end
```


© University of Southampton

- Meaningful names encode design decisions and reduce onboarding time.
- Comments explain *why* (link to requirement or risk mitigation) not *what*.
- Cross-reference tests: `// Verified by tests/integration/gateway_checksum.spec`.

Adopt published standards (e.g., Google Style Guides, PEP 8, MISRA C) that balance readability, tooling compatibility, and domain constraints.

Linting (clang-tidy, pylint, eslint) enforces these conventions automatically, catching deviations and reinforcing collective discipline.

Google emphasizes readability and consistency across teams; PEP 8 prioritizes Pythonic idioms; MISRA enforces strict safety rules for embedded systems.

© University of Southampton

Compare standards

```python
# Google: explicit docstring + type hints
def compute_area(radius: float) -> float:
    """Return area of a circle."""
    if radius < 0:
        raise ValueError("radius must be non-negative")
    return math.pi * radius * radius
```

```python
# PEP 8: snake_case, minimal comments
def compute_area(radius):
    if radius < 0:
        raise ValueError("radius must be non-negative")
    return math.pi * radius * radius
```

```c
/* MISRA C style */
float compute_area(const float radius)
{
    if (radius < 0.0F)
    {
        report_error(RADIUS_NEGATIVE);
        return 0.0F;
    }
    return (float)(PI * radius * radius);
}
```


© University of Southampton

Coupling vs Cohesion

Cohesion: Keep modules focused on a single responsibility so changes remain localized and intent stays obvious.

Coupling: The tighter the links between modules, the more resistance you create to change—treat it like a stiff joint that slows every motion.

Target state: High cohesion + low coupling emerges from clear boundaries, dependency inversion, and well-documented API contracts.

© University of Southampton

Interface Contracts & Protocol Design

Define the intent: Capture payload schemas, timing guarantees, and error codes as you would clauses in a legal agreement.

Version discipline: Maintain compatibility matrices (e.g., “cloud v3 requires firmware ≥1.4”) so change impact is obvious.

Testing hooks: Contract tests in CI prevent silent drift and prove each interface still satisfies stakeholder acceptance criteria.

© University of Southampton

Error Handling Models

Fail-fast: Abort on unrecoverable faults (e.g., cryptographic mismatch) so safety and security requirements never degrade silently.

Fail-safe / degraded: Gateways buffer locally when the cloud is unreachable and raise alerts to keep stakeholders informed.

Compensation: Saga-style workflows roll back partial actions; thorough logging keeps traceability intact.

Risk: Silent retry storms can overload dependencies—use circuit breakers and metrics tied to non-functional requirements.

© University of Southampton

```
# Good: explicit failure path
if sensorPacket.hasValidSignature() then
    telemetryChannel.transmit(sensorPacket)
else
    raise SecurityException("Invalid signature", packetId)
    audit.logIncident(packetId, "signature_mismatch")
end
```

Good error handling surfaces the fault, halts unsafe behaviour, and records context for traceability.

```
# Bad: silent retry loop
try
    transmit(sensorPacket)
catch Exception
    retry()  # no backoff, no logging
end
```

Bad handling swallows the exception, retries blindly, and hides evidence needed for audits or recovery.

© University of Southampton

Performance vs Readability Trade-offs

Context matters: Firmware often prioritises deterministic timing, while cloud services can emphasise readability and observability.

Evidence first: Profile before sacrificing clarity, and document decisions in ADRs linked to commits.

Benchmark discipline: Compare naive vs optimised implementations and capture CPU/memory plus maintainability notes.

© University of Southampton

Design Patterns for Modularity

Ports & Adapters: Separate domain logic from infrastructure so systemic concerns stay isolated.

Observer/Event streams: Model feedback loops and integrate monitoring hooks directly into the architecture.

State machines: Capture firmware or gateway behaviour with explicit transitions tied to requirements.

© University of Southampton

Dependencies & Integration Strategies

Version hygiene: Pin dependencies and document compatibility so everyone knows where friction may occur.

Governance: Add dependency review steps in CI to surface transitive vulnerabilities or licensing red flags.

Integration layers: Use API gateways/adapters to decouple hardware timing constraints from cloud scaling needs.

Failure mode: Shared libraries updated without gateway rebuilds; mitigate via automated integration tests + release gates.

© University of Southampton

Version Control Workflows

Git-Flow: Feature + release branches suit hardware-in-the-loop sign-offs and scheduled drops.

Trunk-based: Short-lived branches and feature flags enable continuous delivery if automation is strong.

Traceability links: Tie commits to requirement IDs and leverage points so socio-technical effects stay visible.

© University of Southampton

CI/CD Gates & Systemic Innovation Labs

Safe-to-fail loops: Treat CI pipelines as innovation labs where teams iterate with rapid feedback.

Gate sequence: Linting → unit tests → integration tests → hardware-in-the-loop → staging deploy keeps quality layered.

Socio-technical checks: Embed accessibility scans, security reviews, and stakeholder sign-offs alongside technical gates.

© University of Southampton

Summary

- Source code is a living design document—systemic mindsets (holism, feedback loops, giga-maps) and requirement IDs keep intent explicit.
- Informed decomposition: high cohesion, low coupling, and explicit interface contracts let each layer evolve independently.
- Readability, published standards, and linting make code auditable; explicit error models (fail-fast, fail-safe, compensation) surface faults instead of hiding them.
- Profile before trading clarity for speed; patterns (ports & adapters, observers, state machines) preserve modularity.
- Pinned dependencies, traceable commits, and layered CI/CD gates turn the pipeline into governance from requirement to deployment.

© University of Southampton